

Firestore Data Fetching & Pagination Optimization Audit

Comprehensive Architectural Audit of 22 Tables & Lists, Query-Level Cursor Pagination Design, Real-Time Listener Optimization, and Phased Implementation Plan

AUDIT TARGET

fair-fly (Frontend) & fly-api (Backend)

AUDIT DATE

September 27, 2026

AUDITOR

Lead Systems & AI Solutions Architect

STATUS

Audit Completed — Plan Awaiting Approval

1. Executive Summary & Impact Analysis

A comprehensive code audit was conducted across every React component, custom hook, service module, and Express backend route in the **FairFly** application to examine data fetching and pagination behavior.

The audit uncovered a **systemic pattern of full-collection fetching paired with client-side JavaScript slicing** (`array.slice((page - 1) * pageSize, page * pageSize)`). Out of 22 inspected data tables, lists, and navigation listeners:

20 / 22

TABLES FETCHING ENTIRE COLLECTIONS

91%

FRONTEND SLICING RATE

8

UNCONSTRAINED ONSNAPSHOT LISTENERS

90-95%

PROJECTED READ/BANDWIDTH REDUCTION

Critical Architectural Bottlenecks Discovered:

- Unconstrained OperatorProvider & AdminProvider** : Both context providers attach real-time `onSnapshot(collection(firestore, targetCollection))` with no query limits or where filters. An operator viewing their branch appointments or services downloads *every document from every branch in the entire company* and filters them client-side in React `useMemo`.
- Detail Page Full Collection Subscriptions**: In `OperatorServiceProcedure.jsx`, viewing a single active service by ID currently wraps the page in `<OperatorProvider targetCollection="activeServices">`, downloading the *entire activeServices collection* just to execute `activeServices.find(s => s.id === id)`.
- Navigation Layout Polling**: `AdminLayout.jsx` and `OperatorLayout.jsx` subscribe to entire collections (including all `users` with role `operator` or `client`) solely to compute badge counter numbers like `openTickets`, `pendingApps`, and `clientCount`.
- Redundant Dual Listeners**: Several views (e.g. `OperatorsContent.jsx`, `OperatorTicketsContent.jsx`, and `Services.jsx`) fetch the entire dataset via REST API while simultaneously opening an unconstrained Firestore `onSnapshot` listener, doubling network load and Firestore read charges.

2. Complete Table & List Pagination Inventory

The following audit matrix categorizes all 22 data presentation views, their data-fetching pipeline, existing pagination mechanism, collection-level impact, and targeted refactoring strategy:

Component / Feature	Current Fetch Strategy	Current Pagination	Fetches All?	Recommended Change
Admin Clients ClientsContent.jsx	<code>fetchClients() → GET /api/clients → db.collection('users').where('role','==','client').get()</code>	Frontend <code>.slice()</code> (pageSize: 8)	YES (ALL CLIENTS)	Implement query-level pagination on backend with <code>limit</code> and Firestore cursor / offset; debounce search and push filters to query.
Admin Operators OperatorsContent.jsx	Dual Fetch: <code>fetchOperators()</code> REST + <code>onSnapshot(users where role=='operator')</code>	Frontend <code>.slice()</code> (pageSize: 8)	YES (TWICE!)	Remove redundant REST call; use single Firestore query with <code>limit(pageSize)</code> and cursor pagination <code>startAfter()</code> .
Admin Operator Detail OperatorDetailPage.jsx	Analytics REST + <code>onSnapshot(collection('activeServices'))</code> filtered in JS by <code>operatorId</code>	Frontend <code>.slice(0, 5)</code> for tickets	YES (ALL SERVICES)	Constrain <code>activeServices</code> query to <code>where('operatorId', '==', id)</code> with <code>limit(10)</code> ; remove global collection scan.
Admin Dashboard Logs AdminDashboard.jsx	<code>onSnapshot(query(admin-logs, orderBy('timestamp','desc'), limit(5)))</code>	Firestore <code>limit(5)</code>	NO (PROPER)	Already Compliant: Uses query-level <code>limit(5)</code> . Retain real-time listener.
Admin Logs Modal AdminLogsModal.jsx	Modal open triggers <code>onSnapshot(admin-logs, orderBy('timestamp','desc'))</code> (No limit!)	Frontend <code>.slice()</code> (pageSize: 10)	YES (ALL LOGS)	Replace with cursor-paginated Firestore query using <code>limit(10)</code> , <code>startAfter()</code> , and forward/back cursor history.
Admin Franchise Apps FranchiseContent.jsx	<code>AdminProvider → onSnapshot(collection('franchiseApplications'))</code>	Frontend <code>.slice()</code> (pageSize: 8)	YES (ALL APPS)	Refactor to cursor-paginated Firestore query <code>orderBy('submittedAt','desc')</code> , <code>limit(8)</code> , <code>startAfter()</code> .
Admin Inquiry History HistoryContent.jsx	<code>AdminProvider → onSnapshot(collection('inquiries'))</code>	Frontend <code>.slice()</code> (pageSize: 10)	YES (ALL INQUIRIES)	Convert to REST GET endpoint with server-side pagination, or Firestore cursor query with <code>limit(10)</code> and <code>startAfter()</code> .
Admin Qualifications QualificationsContent.jsx	<code>AdminProvider → onSnapshot(collection('qualificationApplications'))</code>	Frontend <code>.slice()</code> (pageSize: 8)	YES (ALL APPS)	Query with <code>where('status','==',filter)</code> (when active), <code>orderBy('createdAt','desc')</code> , <code>limit(8)</code> , <code>startAfter()</code> .
Admin Quick Links QuickLinksContent.jsx	<code>AdminProvider → onSnapshot(collection('quickLinks'))</code>	Frontend <code>.slice()</code> (pageSize: 8)	YES (ALL LINKS)	Dataset is small (~20 items); convert to cached REST GET or add Firestore <code>limit(8)</code> if scale demands.
Admin Resources Library ResourcesContent.jsx	<code>fetchResources() → GET /api/resources → db.collection('resources').get()</code>	Frontend <code>.slice()</code> (pageSize: 8)	YES (ALL FILES)	Add <code>limit</code> and <code>page</code> parameters to <code>/api/resources</code> ; execute bounded Firestore query on backend.
Admin Services Catalog ServiceContent.jsx	<code>AdminProvider → onSnapshot(collection('services'))</code>	Frontend <code>.slice()</code> (pageSize: 8)	YES (ALL SERVICES)	Query with <code>orderBy('name')</code> , <code>limit(8)</code> , <code>startAfter()</code> ; debounce search.
Admin Support Tickets TicketsContent.jsx	Redundant <code>AdminProvider</code> + <code>fetchTickets() → GET /api/tickets</code>	Frontend <code>.slice()</code> (pageSize: 5)	YES (TWICE!)	Remove unused <code>AdminProvider</code> wrapper; add <code>limit(5)</code> and cursor pagination to <code>GET /api/tickets</code> .
Admin Workflow Templates AdminWorkflowTemplates.jsx	<code>AdminProvider → onSnapshot(collection('workflowTemplates'))</code>	Frontend <code>.slice()</code> (pageSize: 8)	YES (ALL TEMPLATES)	Add query limit(8) with <code>orderBy('createdAt','desc')</code> and cursor pagination.
Admin Layout Counters AdminLayout.jsx	<code>onSnapshot</code> on entire <code>services</code> and <code>users</code> (operators+clients)	No Pagination (Count Only)	YES (MASSIVE!)	Replace collection streaming with Firestore <code>getCountFromServer()</code> aggregation queries or backend counter doc.
Operator Appointments OperatorAppointments.jsx	<code>OperatorProvider → onSnapshot(collection('appointments'))</code>	Client filter + <code>.slice()</code> (pageSize: 5)	YES (ALL BRANCHES!)	Scope query: <code>where('branchUid','==',user.uid)</code> , <code>orderBy('createdAt','desc')</code> , <code>limit(5)</code> , <code>startAfter()</code> .
Operator Dashboard OperatorDashboard.jsx	<code>OperatorProvider → onSnapshot(collection('activeServices'))</code>	Client filter + <code>.slice()</code> (pageSize: 5)	YES (ALL BRANCHES!)	Scope query: <code>where('operatorId','==',user.uid)</code> , <code>orderBy('startedAt','desc')</code> , <code>limit(5)</code> .
Operator Service Detail OperatorServiceProcedure.jsx	<code>OperatorProvider → onSnapshot(collection('activeServices'))</code>	Single item <code>.find(s => s.id)</code>	YES (FULL COLLECTION!)	Critical Fix: Remove <code>OperatorProvider</code> ; use single document listener: <code>onSnapshot(doc(firestore, 'activeServices', id))</code> .
Operator Quotations OperatorQuotations.jsx	<code>OperatorProvider → onSnapshot(collection('quotations'))</code>	Client filter + <code>.slice()</code> (pageSize: 5)	YES (ALL BRANCHES!)	Scope query: <code>where('branchUid','==',user.uid)</code> , <code>orderBy('createdAt','desc')</code> , <code>limit(5)</code> , <code>startAfter()</code> .
Operator Inquiries OperatorInquiryForms.jsx	<code>OperatorProvider → onSnapshot(collection('inquiries'))</code>	Client filter + <code>.slice()</code> (pageSize: 5)	YES (ALL BRANCHES!)	Scope query: <code>where('branchUid','==',user.uid)</code> .

Component / Feature	Current Fetch Strategy	Current Pagination	Fetches All?	Recommended Change
				<code>orderBy('createdAt','desc') , limit(5) , startAfter() .</code>
Operator Tickets OperatorTicketsContent.jsx	Redundant <code>OperatorProvider</code> + <code>fetchTickets()</code> → GET <code>/api/tickets</code>	Frontend <code>.slice()</code> (pageSize: 8)	YES (TWICE!)	Remove redundant <code>OperatorProvider</code> ; pass <code>limit=8</code> and cursor to <code>ticketService.fetchTickets</code> .
Operator Layout Counters OperatorLayout.jsx	<code>onSnapshot</code> on entire <code>activeServices</code> , <code>appointments</code> , and <code>tickets</code>	No Pagination (Count Only)	YES (3 COLLECTIONS!)	Replace full collection listeners with scoped <code>where('branchUid' , '==', user.uid)</code> using <code>getCountFromServer()</code> .
Client Marketplace ClientServicesMarketplace.jsx	<code>fetchServices()</code> REST (downloads all active services into React state)	Frontend <code>.slice(0, visibleCount)</code> load more	YES (ALL SERVICES)	Implement server-side cursor pagination with <code>limit=6</code> ; request next page upon reaching sentinel scroll element.
Notifications Drawer NotificationContext.jsx	<code>onSnapshot(notifications where recipientUid=user.uid, limit(50))</code>	Firestore <code>limit(50)</code>	NO (PROPER)	Already Compliant: Properly scoped and limited to 50 documents.
Announcements Feed AnnouncementsPage.jsx	<code>onSnapshot(announcements, orderBy('createdAt','desc'))</code> (No limit)	None (Renders all)	YES (ALL POSTS)	Add <code>limit(10)</code> with cursor-based "Load older announcements" using <code>startAfter()</code> .

3. Detailed Analysis of the 6 Core Audit Inquiries

1. Which tables already use proper Firestore-side pagination?

- **Admin Dashboard Activity Stream (AdminDashboard.jsx)**: Correctly uses `query(collection(firestore, 'admin-logs'), orderBy('timestamp', 'desc'), limit(5))` . Limits reads to 5 rows directly at the database engine.
- **User Notifications Center (NotificationContext.jsx)**: Correctly uses `query(collection(firestore, 'notifications'), where('recipientUid', '==', user.uid), limit(50))` . Limits reads to 50 rows.

2. Which tables are doing frontend-only pagination?

20 out of 22 inspected views download the complete dataset into client memory and paginate using `.slice(start, start + pageSize)` . These include all primary management screens: Admin Clients, Admin Operators, Admin Franchise Applications, Admin Inquiries, Admin Qualifications, Admin Resources, Admin Services, Admin Tickets, Admin Templates, Admin Logs Modal, Operator Appointments, Operator Dashboard Services, Operator Inquiries, Operator Quotations, Operator Tickets, Operator Services, and the Client Services Marketplace.

3. Which tables use onSnapshot() and whether they subscribe to too many documents?

The most severe bandwidth vulnerabilities stem from **unconstrained real-time subscriptions**:

- `OperatorProvider` in `fair-fly/src/context/OperatorContext.jsx` : Listens to raw collections (`appointments` , `activeServices` , `inquiries` , `quotations` , `tickets`) without a `where('branchUid', '==', operatorUid)` clause. A single branch operator receives *all data across all branches* in real-time.
- `OperatorServiceProcedure.jsx` : Uses `OperatorProvider` `targetCollection="activeServices"` , downloading the *entire company's active services database* just to view 1 document.
- `AdminLayout.jsx` and `OperatorLayout.jsx` : Subscribe to `users` (all operators and clients), `services` , `appointments` , and `tickets` just to calculate badge numbers, maintaining active persistent WebChannel streams for thousands of unnecessary documents.
- `AdminDashboard.jsx` (Audit Logs Modal): Subscribes to *every admin action log ever recorded* without limit whenever the modal is opened.

4. Any Firestore Composite Indexes required?

When Firestore combines equality filters (`where`) with ordering (`orderBy`) and range cursors (`startAfter`), composite indexes are strictly mandatory. Since no `firestore.indexes.json` currently exists, the following composite indexes must be deployed:

```
// Target firestore.indexes.json additions:
{
  "indexes": [
    {
      "collectionGroup": "appointments",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "branchUid", "order": "ASCENDING" },
        { "fieldPath": "createdAt", "order": "DESCENDING" }
      ]
    },
    {
      "collectionGroup": "appointments",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "branchUid", "order": "ASCENDING" },
        { "fieldPath": "status", "order": "ASCENDING" },
        { "fieldPath": "createdAt", "order": "DESCENDING" }
      ]
    },
    {
      "collectionGroup": "activeServices",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "operatorId", "order": "ASCENDING" },
        { "fieldPath": "startedAt", "order": "DESCENDING" }
      ]
    },
    {
      "collectionGroup": "activeServices",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "operatorId", "order": "ASCENDING" },
        { "fieldPath": "status", "order": "ASCENDING" },
        { "fieldPath": "startedAt", "order": "DESCENDING" }
      ]
    },
    {
      "collectionGroup": "quotations",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "branchUid", "order": "ASCENDING" },
        { "fieldPath": "createdAt", "order": "DESCENDING" }
      ]
    },
    {
      "collectionGroup": "inquiries",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "branchUid", "order": "ASCENDING" },
        { "fieldPath": "createdAt", "order": "DESCENDING" }
      ]
    },
    {
      "collectionGroup": "tickets",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "operatorId", "order": "ASCENDING" },
        { "fieldPath": "createdAt", "order": "DESCENDING" }
      ]
    },
    {
      "collectionGroup": "admin-logs",
      "queryScope": "COLLECTION",
```

```
"fields": [
  { "fieldPath": "actionType", "order": "ASCENDING" },
  { "fieldPath": "timestamp", "order": "DESCENDING" }
]
}
```

5. Any filtering/search requirements that conflict with proposed queries?

- **Client-Side Substring Search Conflict:** Existing search bars evaluate multi-field JavaScript substrings: `(item.name.toLowerCase().includes(q) || item.email.includes(q) || item.phone.includes(q))`. Firestore native queries cannot perform arbitrary multi-field partial substring searches across unread documents without reading them.
- **Conflict Resolution Strategy:**
 1. For REST API tables (Clients, Resources, Tickets), transfer search handling to the backend endpoint with bounded Firestore retrieval and limit bounds.
 2. For Firestore real-time tables, apply equality status/branch filters and deterministic ordering in Firestore, and pair text searches with a dedicated search prefix or debounced server-side query with a 50-item hard limit safeguard.
- **Global KPI Aggregations vs Page Size:** AlertBars compute counts (e.g. `applications.filter(a => a.status === 'pending').length`). If only 8 docs are fetched, the counter would read 8 instead of the database total.
Solution: Use Firestore's lightweight `getCountFromServer()` aggregation queries, which calculate exact document counts on the server with negligible bandwidth and 1 read charge per 1,000 index items.

6. Any places where changing the query could affect existing functionality?

- **Cursor Sliding on Real-Time Mutations:** If a table uses real-time `onSnapshot` with `startAfter(lastDoc)`, a newly inserted document at the top of the collection could push boundaries and cause duplicate row displays on page 2. For static/administrative tables, moving from `onSnapshot` to paginated REST GET endpoints eliminates this instability.
- **Back-Page Navigation:** Unlike SQL `OFFSET`, Firestore cursors are unidirectional. Navigating backwards requires maintaining a page cursor stack in React state: `pageSnapshots = [docPage1First, docPage2First, ...]`.

4. Phased Implementation Plan & Migration Architecture

The refactoring will be executed across 5 logical phases, adhering to the `AGENTS.md` rule: "Reads should use `onSnapshot` for real-time data, use `query` on snapshots to prevent unnecessary frontend filtering, make `GET` requests for data that doesn't require real-time data".

Implementation Rules & Safety Guarantees:

- No code refactoring will begin until this plan and audit matrix are approved.
- Each phase will be verified independently with zero breaking changes to existing UI components (`DataTable` , `Pagination` , `FilterChipGroup`).
- Page 1 will strictly request only the designated page size (5 to 10 documents).
- Loading, empty, error, and filtering states will be preserved verbatim.

PHASE	SCOPE & TARGET FILES	TECHNICAL IMPLEMENTATION DETAILS	VERIFICATION CRITERIA
Phase 1: Critical Leak Fixes	<code>OperatorServiceProcedure.jsx</code> <code>AdminLayout.jsx</code> <code>OperatorLayout.jsx</code> <code>OperatorDetailPage.jsx</code>	- Replace <code>OperatorProvider</code> in <code>OperatorServiceProcedure.jsx</code> with direct single-doc listener: <code>onSnapshot(doc(firestore, 'activeServices', id))</code>. - Refactor navigation badge listeners in <code>AdminLayout</code> & <code>OperatorLayout</code> to use <code>getCountFromServer()</code> aggregation queries instead of streaming thousands of user & service docs. - Constrain <code>OperatorDetailPage</code> <code>activeServices</code> listener with <code>where('operatorId', '==', id)</code>.	Detail page reads drop from entire collection to 1 doc. Layout streams 0 documents into memory.
Phase 2: Operator Portal Tables	<code>OperatorAppointments.jsx</code> <code>OperatorDashboard.jsx</code> <code>OperatorQuotations.jsx</code> <code>OperatorInquiryForms.jsx</code> <code>OperatorContext.jsx</code>	- Upgrade <code>OperatorProvider</code> to accept query constraints (<code>operatorUid</code> , <code>pageSize</code> , <code>cursor</code>). - Refactor Operator tables to execute queries scoped by <code>where('branchUid', '==', user.uid)</code> , ordered deterministically by timestamp, with <code>limit(pageSize)</code> . - Implement cursor stack (<code>startAfter(lastVisibleDoc)</code>) for smooth page navigation. - Use <code>getCountFromServer()</code> for KPI chips and total page count calculation.	Network inspection confirms Page 1 requests exactly 5 documents. Operator only receives own branch records.
Phase 3: Admin Portal Real-Time Tables	<code>FranchiseContent.jsx</code> <code>QualificationsContent.jsx</code> <code>ServiceContent.jsx</code> <code>AdminLogsModal.jsx</code> <code>AdminWorkflowTemplates.jsx</code>	- Upgrade <code>AdminProvider</code> to support cursor pagination with <code>orderBy</code> , <code>limit(pageSize)</code> , and <code>startAfter(cursor)</code> . - Refactor <code>AdminLogsModal.jsx</code> to fetch logs in batches of 10 with cursor-based pagination instead of listening to all historical logs. - Deploy composite indexes in <code>firestore.indexes.json</code> for status + timestamp sorting.	Admin tables read 8-10 documents per page. Modal loads instantly without memory spikes.
Phase 4: Backend REST API Pagination	<code>fly-api/src/controllers/clientController.js</code> <code>fly-api/src/controllers/resourceController.js</code> <code>fly-api/src/controllers/ticketController.js</code> <code>fair-fly/src/pages/Admin/AdminClients</code> <code>fair-fly/src/pages/Admin/AdminResources</code> <code>fair-fly/src/pages/Admin/AdminTickets</code>	- Add <code>page</code> , <code>limit</code> , and <code>search</code> parameters to <code>GET /api/clients</code> , <code>GET /api/resources</code> , and <code>GET /api/tickets</code> . - Implement query bounds and return <code>{ data, total, page, totalPages }</code> . - Remove duplicate <code>onSnapshot</code> listeners in <code>AdminTickets</code> and <code>OperatorTickets</code> . - Connect frontend <code>DataTable</code> and <code>Pagination</code> to server-driven pagination state.	Backend response payloads shrink from entire databases to 8 records per HTTP call.
Phase 5: Client Store & Verification	<code>ClientServicesMarketplace.jsx</code> <code>AnnouncementsPage.jsx</code> <code>firestore.indexes.json</code>	- Implement server-side batched retrieval (<code>limit: 6</code>) for Client Services Store with infinite scroll trigger. - Add <code>limit(10)</code> to <code>AnnouncementsPage</code> with "Load older announcements" cursor. - Conduct full automated performance & security regression testing.	Total initial bundle transfer and Firestore document reads confirmed reduced by >90%.